

CURSO PRATICO

Do zero ao Sangalli Gestao

Como construir, na pratica, o sistema web de gestao da Oficina Funilaria Sangalli - do primeiro arquivo ao deploy em producao.

Stack: React + TypeScript + Tailwind (frontend) | Node.js + Express + Prisma (backend) | PostgreSQL / Supabase (banco de dados)

Baseado no codigo real do projeto AutoControl / Sangalli Gestao - 2026.

Sumario do curso

1. Visao geral: o que vamos construir
2. Preparando o ambiente de desenvolvimento
3. Criando o backend do zero (Node + Express + TypeScript)
4. Modelando o banco de dados com Prisma
5. Organizando a API em camadas
6. Autenticacao com JWT
7. Construindo o primeiro CRUD: Clientes
8. Tratamento de erros centralizado
9. Repetindo o padrao para os demais recursos
10. Funcionalidades especiais do backend
11. Seguranca da API
12. Testes automatizados
13. Criando o frontend do zero (React + Vite + Tailwind)
14. Consumindo a API no frontend
15. Construindo as telas do sistema
16. Transformando em PWA
17. Publicando em producao (deploy)
18. Recapitulando o que voce aprendeu

Como usar este material: cada modulo pode ser seguido na ordem, digitando e rodando os trechos de codigo no seu proprio computador. Os exemplos usam exatamente os nomes de arquivos e trechos de codigo do projeto Sangalli Gestao, para que voce possa comparar com o repositorio real a qualquer momento.

Visao geral: o que vamos construir

Antes de escrever a primeira linha de codigo, entenda o mapa completo.

O Sangalli Gestao e um sistema web para uma oficina de funilaria e mecanica: ele substitui anotacoes em papel por cadastro de clientes e veiculos, ordens de servico com status (do diagnostico ate a entrega), orcamento automatico e uma pagina publica onde o cliente acompanha o proprio veiculo sem precisar de login.

A arquitetura em 3 camadas

Todo sistema web como este e dividido em tres partes que conversam entre si por HTTP (a mesma linguagem que seu navegador usa para abrir qualquer site):

Camada	O que e	Tecnologia usada aqui
Frontend	O que o usuario ve e clica, roda no navegador	React + TypeScript + Tailwind CSS
Backend (API)	Recebe pedidos, aplica regras, fala com o banco	Node.js + Express + Prisma
Banco de dados	Onde os dados ficam guardados de verdade	PostgreSQL (hospedado no Supabase)

O frontend nunca fala direto com o banco de dados - ele sempre passa pelo backend, que decide o que e permitido fazer. Essa separacao e o motivo pelo qual vamos construir dois projetos distintos (uma pasta backend/ e uma pasta frontend/), cada um com seu proprio package.json.

COMO USAR ESTE CURSO

Ao longo do curso voce vai literalmente recriar a estrutura de pastas do projeto real. Sempre que aparecer um caminho como backend/src/routes/clientRoutes.ts, e o arquivo correspondente do repositorio Sangalli Gestao.

Preparando o ambiente de desenvolvimento

O que instalar antes de começar

Passo 1 - Instalar o Node.js (versao 20 ou superior)

É o motor que executa JavaScript/TypeScript fora do navegador - tanto o backend quanto as ferramentas do frontend (Vite) rodam sobre ele. Baixe em nodejs.org e confirme a instalação com 'node -v' no terminal.

Passo 2 - Instalar o Git

Ferramenta de controle de versão: grava o histórico de mudanças do código e permite subir o projeto para o GitHub.

Passo 3 - Criar uma conta no GitHub

É onde o repositório remoto do projeto vai morar, permitindo colaboração entre os integrantes do grupo.

Passo 4 - Criar um projeto no Supabase

O Supabase fornece um banco PostgreSQL gratuito na nuvem, sem precisar instalar banco de dados na sua máquina. No painel, você vai encontrar a string de conexão (DATABASE_URL) que o Prisma vai usar.

Passo 5 - Instalar um editor de código

O projeto real foi desenvolvido no Visual Studio Code, com extensões de ESLint e Prisma para facilitar a leitura do código.

```
node -v
npm -v
git --version
```

Se os três comandos acima responderem com um número de versão (e não um erro de 'comando não encontrado'), o ambiente está pronto.

Criando o backend do zero

A primeira pasta do projeto: backend/, com Node.js, Express e TypeScript.

Passo 1 - Criar a pasta e iniciar o projeto Node

Dentro da pasta raiz do projeto (ex: AutoControl/), crie a pasta backend e inicialize o package.json.

```
mkdir backend && cd backend
npm init -y
```

Passo 1 - Instalar as dependencias principais

Express é o framework que recebe as requisições HTTP; TypeScript adiciona tipagem; ts-node-dev roda o código TypeScript direto, reiniciando o servidor a cada alteração salva (como um 'live reload' para a API).

```
npm install express cors dotenv
npm install -D typescript ts-node-dev @types/node @types/express
```

Passo 1 - Configurar o TypeScript

O arquivo tsconfig.json diz ao compilador onde estão os arquivos-fonte e para onde deve gerar o código JavaScript final (usado em produção).

backend/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "rootDir": "src",
    "outDir": "dist",
    "strict": true,
    "esModuleInterop": true
  }
}
```

Passo 1 - Criar o primeiro servidor, so para provar que funciona

Antes de qualquer regra de negócio, é essencial ver o servidor respondendo. No projeto real, essa responsabilidade é dividida em dois arquivos: app.ts (monta as regras da aplicação) e server.ts (liga o servidor numa porta).

backend/src/server.ts

```
import 'dotenv/config';
import { createApp } from './app';

const app = createApp();
const port = process.env.PORT ? Number(process.env.PORT) : 3333;

app.listen(port, () => {
  console.log(`Sangalli Gestao API rodando em http://localhost:${port}`);
});
```

backend/src/app.ts (primeira versão, simplificada)

```
import express from 'express';
import cors from 'cors';

export function createApp() {
  const app = express();
```

```
app.use(cors());
app.use(express.json());

app.get('/api/health', (_req, res) => res.json({ status: 'ok' }));

return app;
}
```

Rode 'npx ts-node-dev --respawn --transpile-only src/server.ts' e abra <http://localhost:3333/api/health> no navegador. Se aparecer {"status":"ok"}, seu backend esta de pe. Esse comando e o mesmo que o script 'npm run dev' do projeto real executa.

Modelando o banco de dados com Prisma

O Prisma é um ORM (Object-Relational Mapper): ele permite descrever as tabelas do banco em um arquivo de texto (schema.prisma) e depois manipular esses dados usando funções JavaScript, sem escrever SQL na mão.

Passo 1 - Instalar o Prisma e iniciar

O comando 'prisma init' cria a pasta prisma/ com um schema.prisma vazio e um arquivo .env para a string de conexão com o banco.

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

Passo 1 - Descrever as tabelas no schema.prisma

Cada 'model' vira uma tabela no banco. Os campos com @relation descrevem como as tabelas se conectam (ex: um Cliente tem vários Veículos).

backend/prisma/schema.prisma (modelo simplificado de Cliente)

```
model Client {
  id      String  @id @default(uuid())
  name    String
  cpf     String  @unique
  phone   String
  email   String?
  createdAt DateTime @default(now()) @map("created_at")

  vehicles  Vehicle[]
  workOrders WorkOrder[]

  @@map("clients")
}
```

NOTA

No schema.prisma real do projeto há 10 tabelas: users, clients, vehicles, services, parts, work_orders, work_order_services, work_order_parts, work_order_photos, work_order_status_history e payments. Cada uma segue o mesmo padrão: campos, depois relacionamentos.

Passo 1 - Aplicar o schema no banco de dados de verdade

O comando 'migrate dev' compara o schema com o banco e gera/aplica o SQL necessário para deixá-los iguais - e cria um arquivo de migração versionado.

Passo 2 - Gerar o Prisma Client

É o 'tradutor' que você vai importar no código para consultar o banco: prisma.client.findMany(), prisma.client.create(), etc.

```
npx prisma migrate dev --name init
npx prisma generate
```

A partir daqui, sempre que você alterar o schema.prisma, repita esses dois comandos para manter o banco e o código sincronizados.

Organizando a API em camadas

Por que dividir o código em routes, controllers, middlewares e validators.

Poderíamos escrever toda a lógica dentro de `app.ts`, mas isso viraria um arquivo gigante e difícil de manter conforme o sistema cresce. O projeto Sangalli Gestão usa uma arquitetura em camadas, onde cada arquivo tem uma única responsabilidade:

Pasta	Responsabilidade
<code>routes/</code>	Define quais URLs existem e qual função atende cada uma
<code>controllers/</code>	Executa a ação: lê a requisição, fala com o banco, monta a resposta
<code>middlewares/</code>	Código que roda ANTES do controller (autenticação, log, etc.)
<code>validators/</code>	Regras de formato dos dados de entrada (com a biblioteca Zod)
<code>utils/</code>	Funções auxiliares reutilizáveis (tratamento de erro, wrappers)

Passo 1 - Criar o arquivo central de rotas

Ele importa as rotas de cada recurso e as registra sob um prefixo de URL.

`backend/src/routes/index.ts`

```
import { Router } from 'express';
import clientRoutes from './clientRoutes';
import vehicleRoutes from './vehicleRoutes';

const router = Router();

router.get('/health', (_req, res) => res.json({ status: 'ok' }));
router.use('/clientes', clientRoutes);
router.use('/veiculos', vehicleRoutes);

export default router;
```

E em `app.ts`, esse roteador central é conectado sob o prefixo `/api`: `'app.use("/api", routes)'`. Assim, uma rota de clientes fica acessível em `'http://localhost:3333/api/clientes'`.

Autenticacao com JWT

Como o sistema reconhece um usuario logado em cada requisicao.

JWT (JSON Web Token) e um 'crachá digital' que o servidor entrega depois de um login valido. Ele carrega quem e o usuario e vem assinado digitalmente, de forma que ninguem consiga forja-lo sem conhecer o segredo do servidor.

6.1 Guardando senhas com seguranca

Senhas nunca sao guardadas como texto puro no banco. O bcrypt transforma a senha em um hash irreversivel antes de salvar, e depois compara o hash na hora do login.

```
npm install bcrypt jsonwebtoken
npm install -D @types/bcrypt @types/jsonwebtoken
```

6.2 A rota de login

backend/src/controllers/authController.ts

```
export const login = asyncHandler(async (req, res) => {
  const { email, password } = loginSchema.parse(req.body);

  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) throw new UnauthorizedError('E-mail ou senha invalidos.');
```

```
  const passwordMatches = await bcrypt.compare(password, user.passwordHash);
  if (!passwordMatches) throw new UnauthorizedError('E-mail ou senha invalidos.');
```

```
  const token = jwt.sign(
    { sub: user.id, role: user.role, name: user.name },
    process.env.JWT_SECRET as string,
    { expiresIn: process.env.JWT_EXPIRES_IN ?? '8h' },
  );

  return res.json({ token, user: { id: user.id, name: user.name, email: user.email } });
});
```

6.3 O middleware que protege as rotas

backend/src/middlewares/auth.ts

```
export function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith('Bearer ')) {
    throw new UnauthorizedError('Token de autenticacao ausente.');
```

```
  }
  const token = authHeader.replace('Bearer ', '');

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET as string);
    next();
  } catch {
    throw new UnauthorizedError('Token invalido ou expirado.');
```

```
  }
}
```

Para proteger um grupo de rotas, basta chamar 'router.use(authenticate)' no topo do arquivo de rotas

daquele recurso - toda rota abaixo dessa linha passa a exigir o token.

SEGURANCA

A variavel JWT_SECRET fica no arquivo .env (nunca no codigo-fonte nem no Git) e deve ser uma string longa e aleatoria. Se ela vaziar, qualquer pessoa consegue forjar tokens validos - por isso .env entra no .gitignore.

Construindo o primeiro CRUD: Clientes

CRUD = Create, Read, Update, Delete - as quatro operacoes basicas de um cadastro.

7.1 Validando os dados de entrada (Zod)

Antes de gravar qualquer coisa no banco, validamos o formato dos dados que chegaram. O Zod descreve as regras de forma declarativa e gera mensagens de erro legíveis automaticamente.

```
npm install zod
```

```
backend/src/validators/clientValidators.ts
```

```
export const createClientSchema = z.object({
  name: z.string().min(2, 'Informe o nome completo.'),
  cpf: z.string().min(11, 'CPF invalido.').max(14, 'CPF invalido.'),
  phone: z.string().min(8, 'Informe um telefone valido.'),
  email: z.string().email('E-mail invalido.').optional().nullable(),
});

export const updateClientSchema = createClientSchema.partial();
```

7.2 O controller: a logica de cada operacao

```
backend/src/controllers/clientController.ts
```

```
export const listClients = asyncHandler(async (req, res) => {
  const clients = await prisma.client.findMany({ orderBy: { name: 'asc' } });
  return res.json(clients);
});

export const createClient = asyncHandler(async (req, res) => {
  const data = createClientSchema.parse(req.body);
  const client = await prisma.client.create({ data });
  return res.status(201).json(client);
});
```

7.3 As rotas: ligando URL a funcao

```
backend/src/routes/clientRoutes.ts
```

```
const router = Router();

router.use(authenticate);

router.get('/', listClients);
router.get('/:id', getClient);
router.post('/', createClient);
router.put('/:id', updateClient);
router.delete('/:id', deleteClient);

export default router;
```

7.4 Testando sem precisar do frontend

Use uma ferramenta como Insomnia, Postman ou a extensao Thunder Client do VS Code para simular requisicoes antes de existir qualquer tela. Primeiro faca login em POST /api/auth/login para pegar um token; depois use esse token no cabeçalho Authorization das demais chamadas.

```
POST http://localhost:3333/api/clientes
```

```
Authorization: Bearer <token do login>
```

```
Content-Type: application/json
```

```
{ "name": "Maria Silva", "cpf": "12345678900", "phone": "17999999999" }
```

Repetindo exatamente esse padrao (validator -> controller -> route) para cada tabela do banco, voce constroi todos os CRUDs do sistema: veiculos, servicos, pecas, ordens de servico e assim por diante.

Tratamento de erros centralizado

Sem um tratamento central, cada controller precisaria de blocos try/catch repetidos, e cada erro (validacao, CPF duplicado, item nao encontrado) responderia num formato diferente para o frontend. A solucao: uma unica porta de saida para qualquer erro.

Passo 1 - Um wrapper para funcoes assincronas

Toda funcao de controller e 'async'. Se ela lancar um erro sem esse wrapper, o Express nao intercepta automaticamente - a aplicacao trava. O asyncHandler resolve isso encaminhando qualquer erro para o Express.

backend/src/utils/asyncHandler.ts

```
export function asyncHandler(handler) {
  return (req, res, next) => {
    handler(req, res, next).catch(next);
  };
}
```

Passo 1 - Um middleware de erro, registrado por ultimo

O Express reconhece uma funcao com 4 parametros (err, req, res, next) como tratador de erros e a chama sempre que algo cai no catch acima.

backend/src/middlewares/errorHandler.ts

```
export function errorHandler(err, req, res, next) {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({ error: err.message });
  }
  if (err instanceof ZodError) {
    return res.status(422).json({ error: 'Dados invalidos.', issues: err.issues });
  }
  if (err instanceof Prisma.PrismaClientKnownRequestError && err.code === 'P2002') {
    return res.status(409).json({ error: 'Ja existe um registro com esse valor unico.' });
  }
  console.error(err);
  return res.status(500).json({ error: 'Erro interno no servidor.' });
}
```

Em app.ts, esse middleware e registrado por ultimo, depois de todas as rotas: 'app.use(errorHandler)'. A ordem importa - middlewares de erro so funcionam se vierem depois das rotas que podem falhar.

Repetindo o padrao para os demais recursos

Com o padrao validator -> controller -> route dominado, o restante do backend e questao de repeticao disciplinada, recurso por recurso:

- Veiculos (vehicleRoutes): sempre vinculados a um cliente (clientId), com placa unica.
- Servicos e Pecas (serviceRoutes, partRoutes): catalogo com preco, usado para montar orcamentos.
- Ordens de Servico (workOrderRoutes): o recurso mais complexo - liga cliente, veiculo, servicos escolhidos, pecas usadas, fotos e pagamentos em um so registro.

O fluxo de status da ordem de servico

Uma ordem de servico segue um caminho fixo, representado por um enum no Prisma - isso evita, por exemplo, que uma OS va direto de 'diagnostico' para 'entregue' sem passar pelas etapas intermediarias:

backend/prisma/schema.prisma

```
enum WorkOrderStatus {  
  EM_DIAGNOSTICO  
  AGUARDANDO_APROVACAO  
  EM_MANUTENCAO  
  EM_FUNILARIA  
  EM_PINTURA  
  EM_TESTE  
  PRONTO  
  ENTREGUE  
  CANCELADO  
}
```

Cada mudanca de status e registrada numa tabela separada, work_order_status_history, guardando quem mudou e quando - isso e o que alimenta a 'linha do tempo' que o cliente ve na pagina publica de acompanhamento.

Funcionalidades especiais do backend

10.1 Pagina publica de acompanhamento, sem login

Cada ordem de servico recebe um publicToken (um UUID aleatorio) no momento em que e criada. Uma rota separada e publica (sem passar pelo middleware authenticate) busca a OS por esse token, e devolve so os dados que o cliente pode ver - escondendo notas internas da oficina.

backend/src/routes/publicRoutes.ts (resumo)

```
router.get('/:token', async (req, res) => {
  const workOrder = await prisma.workOrder.findUnique({
    where: { publicToken: req.params.token },
    include: { client: true, vehicle: true, statusHistory: true, photos: true },
  });
  if (!workOrder) throw new NotFoundError();
  return res.json(workOrder); // internalNotes fica de fora da resposta
});
```

10.2 QR Code

A biblioteca 'qrcode' gera uma imagem a partir do link publico (/acompanhar/:token), permitindo que o cliente escaneie no balcao da oficina em vez de digitar o link.

10.3 Upload de fotos (Cloudinary)

As fotos do veiculo nao ficam no banco de dados - so a URL e o identificador (public_id) sao guardados na tabela work_order_photos. O arquivo em si sobe para o Cloudinary, um servico externo especializado em imagens, via Multer (que recebe o upload) e o SDK do Cloudinary (que envia o arquivo).

10.4 Dashboard e relatorios

Rotas dedicadas (dashboardRoutes, reportRoutes) usam funcoes de agregacao do Prisma (groupBy, aggregate) para calcular faturamento mensal, ordens por status e servicos mais realizados - dados que alimentam os graficos do frontend.

Seguranca da API

Alem da autenticao JWT, a API aplica camadas adicionais de protecao, configuradas centralmente em app.ts:

Ferramenta	O que protege
helmet	Define cabecalhos HTTP de seguranca (evita alguns ataques comuns do navegador)
cors	Permite requisicoes apenas do dominio do frontend (FRONTEND_URL)
express-rate-limit	Limita quantas requisicoes por IP num periodo (evita forca bruta no login)
bcrypt	Senhas nunca ficam em texto puro no banco
zod	Nenhum dado entra no banco sem validacao de formato

backend/src/app.ts (trecho de seguranca)

```
app.use(helmet());
app.use(cors({ origin: process.env.FRONTEND_URL }));

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 20,
  message: { error: 'Muitas tentativas de login. Tente novamente em alguns minutos.' },
});
app.use('/api/auth/login', authLimiter);
```

Testes automatizados

Testes automatizados confirmam que o sistema continua funcionando depois de qualquer alteracao, sem precisar testar manualmente tudo de novo. O backend usa Vitest (executor de testes) junto com Supertest (simula requisicoes HTTP sem precisar do servidor rodando de verdade).

```
npm install -D vitest supertest
```

backend/tests/clients.test.ts (exemplo simplificado)

```
import { describe, it, expect } from 'vitest';
import request from 'supertest';
import { createApp } from '../src/app';

describe('POST /api/clientes', () => {
  it('rejeita criacao sem token de autenticacao', async () => {
    const app = createApp();
    const response = await request(app).post('/api/clientes').send({});
    expect(response.status).toBe(401);
  });
});
```

No frontend, o mesmo Vitest e usado junto com React Testing Library, que simula cliques e digitacao do usuario nos componentes.

Criando o frontend do zero

A segunda pasta do projeto: `frontend/`, com `React`, `Vite`, `TypeScript` e `Tailwind`.

Passo 1 - Criar o projeto com Vite

O Vite é a ferramenta que compila e serve o React durante o desenvolvimento, com recarregamento instantâneo ao salvar um arquivo.

```
npm create vite@latest frontend -- --template react-ts
cd frontend
npm install
```

Passo 1 - Instalar e configurar o Tailwind CSS

Tailwind fornece classes utilitárias (ex: `'flex'`, `'p-4'`, `'text-green-600'`) que você combina direto no HTML/JSX, sem escrever arquivos `.css` separados para cada componente.

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Passo 1 - Organizar a estrutura de pastas

O projeto real separa responsabilidades claramente dentro de `src/`:

Pasta	Conteúdo
<code>pages/</code>	Uma tela inteira do sistema (<code>ClientsPage</code> , <code>DashboardPage...</code>)
<code>components/</code>	Pedacos de interface reutilizáveis (<code>StatCard</code> , <code>StatusBadge...</code>)
<code>contexts/</code>	Estado compartilhado entre várias telas (<code>AuthContext</code>)
<code>services/</code>	Funções que chamam a API (<code>clientService.ts</code> , <code>authService.ts...</code>)
<code>types/</code>	Definições TypeScript compartilhadas (<code>User</code> , <code>Client</code> , <code>WorkOrder...</code>)

Consumindo a API no frontend

14.1 Um cliente HTTP central (axios)

Em vez de repetir a URL base e o cabeçalho de autenticação em cada tela, um único arquivo configura o axios (biblioteca de requisições HTTP) para toda a aplicação.

frontend/src/services/api.ts

```
export const api = axios.create({ baseURL: import.meta.env.VITE_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('autocontrol:token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('autocontrol:token');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  },
);
```

14.2 Guardando quem esta logado (Context API)

O AuthContext expõe, para qualquer tela, se há um usuário logado e as funções login/logout - e persiste o token no localStorage para sobreviver a um F5 na página.

frontend/src/contexts/AuthContext.tsx (resumo)

```
const login = async (email, password) => {
  const response = await loginRequest(email, password);
  localStorage.setItem('autocontrol:token', response.token);
  setToken(response.token);
  setUser(response.user);
};
```

14.3 Bloqueando telas para quem não está logado

frontend/src/components/ProtectedRoute.tsx

```
export function ProtectedRoute() {
  const { isAuthenticated, isLoading } = useAuth();
  if (isLoading) return null;
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  return <Outlet />;
}
```

No React Router, essa checagem envolve um grupo inteiro de rotas de uma só vez, sem precisar repetir a lógica em cada página individualmente.

Construindo as telas do sistema

Com a comunicacao com a API pronta, cada tela segue um padrao parecido: buscar dados ao montar o componente, mostrar em uma tabela/lista, e oferecer um formulario para criar/editar.

frontend/src/pages/ClientsPage.tsx (estrutura simplificada)

```
export function ClientsPage() {
  const [clients, setClients] = useState<Client[]>([]);

  useEffect(() => {
    listClients().then(setClients);
  }, []);

  return (
    <div>
      {clients.map((c) => <ClientRow key={c.id} client={c} />)}
    </div>
  );
}
```

As rotas de tela sao centralizadas em App.tsx, que decide qual componente mostrar para cada URL - e envolve as rotas privadas dentro de ProtectedRoute:

frontend/src/App.tsx (resumo)

```
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/acompanhar/:token" element={<PublicTrackingPage />} />

  <Route element={<ProtectedRoute />>
    <Route element={<AppLayout />>
      <Route path="/dashboard" element={<DashboardPage />} />
      <Route path="/clientes" element={<ClientsPage />} />
      <Route path="/ordens" element={<WorkOrdersPage />} />
      {/* ...demais paginas */}
    </Route>
  </Route>
</Routes>
```

Repare que /login e /acompanhar/:token ficam FORA do ProtectedRoute - fazem sentido sem estar logado, ja que sao, respectivamente, a propria tela de login e a pagina publica do cliente.

O painel (DashboardPage) usa a biblioteca Recharts para desenhar graficos (faturamento mensal, ordens por status) a partir dos dados que vem da rota /dashboard do backend.

Transformando em PWA

PWA (Progressive Web App) permite que o site seja 'instalado' no celular, como um aplicativo, e continue funcionando (parcialmente) sem internet. No projeto, isso é feito com o plugin vite-plugin-pwa.

```
npm install -D vite-plugin-pwa
```

frontend/vite.config.ts (resumo)

```
import { VitePWA } from 'vite-plugin-pwa';

export default defineConfig({
  plugins: [
    react(),
    VitePWA({
      registerType: 'autoUpdate',
      workbox: { globPatterns: ['**/*.js', '**/*.css', '**/*.html', '**/*.svg', '**/*.png', '**/*.ico'] },
    }),
  ],
});
```

Um arquivo manifest.json descreve o nome do app, ícones e cores; o plugin gera automaticamente um Service Worker (um script que roda em segundo plano no navegador) responsável por guardar arquivos em cache.

Publicando em producao (deploy)

Com tudo funcionando localmente, falta colocar o sistema no ar, acessível de qualquer lugar. Cada camada tem um serviço de nuvem próprio:

Camada	Serviço usado	O que você configura
Banco de dados	Supabase	Projeto Postgres gerenciado; a DATABASE_URL vai para as variáveis de ambiente do backend
Backend (API)	Render	Web Service Node; build 'npm install && npm run build'; start 'npm start'
Frontend	Netlify	Build 'npm run build'; publica a pasta dist/
Imagens	Cloudinary	Credenciais também via variáveis de ambiente, nunca no código

REGRA DE OURO

Nenhuma senha, chave de API ou string de conexão deve ir para o repositório Git. Elas ficam no arquivo .env local (que entra no .gitignore) e são cadastradas manualmente no painel de cada serviço de hospedagem.

Depois do primeiro deploy, tanto o Render quanto o Netlify observam o repositório no GitHub: a cada 'git push' na branch principal, uma nova versão é publicada automaticamente.

Recapitulando o que voce aprendeu

Se voce seguiu os 17 modulos anteriores, construiu - na pratica, nao so na teoria - uma aplicacao full-stack completa. Veja o mapa geral do que foi aprendido:

- Como separar um sistema em frontend, backend e banco de dados, e por que essa separacao existe.
- Como modelar dados relacionais com Prisma e aplicar migracoes num banco Postgres real.
- Como organizar uma API REST em camadas (rotas, controllers, middlewares, validators).
- Como autenticacao com JWT funciona de ponta a ponta - do login ao middleware que protege cada rota.
- Como validar dados de entrada e centralizar o tratamento de erros.
- Como construir uma interface React que consome essa API, com autenticacao e rotas protegidas.
- Como transformar o frontend num PWA instalavel.
- Como publicar cada camada num servico de nuvem gratuito.

Proximos passos sugeridos

- Adicionar permissões por perfil de usuario (hoje o sistema tem so o papel ADMIN).
- Implementar notificacoes automaticas (e-mail/WhatsApp) quando o status da OS muda.
- Exportar relatorios em PDF/Excel.
- Escrever mais testes automatizados, cobrindo os fluxos criticos (login, criacao de OS, orcamento).

O melhor jeito de fixar esse conteudo e comparar cada modulo deste curso com o codigo real do repositorio Sangalli Gestao, arquivo por arquivo, e tentar recriar um recurso novo (por exemplo, um cadastro de fornecedores) seguindo exatamente o mesmo padrao.